AWS DATABASE MIGRATION PROJECT

Oracle to Aurora PostgreSQL Using AWS DMS

Hands-on implementation | Full-load migration | Amazon RDS, Aurora PostgreSQL, EC2, AWS DMS

PROJECT OUTCOME	RESULT
Oracle source records	5
PostgreSQL target records	5
Migration method	AWS DMS - full load
Reported outcome	Five customer records successfully copied

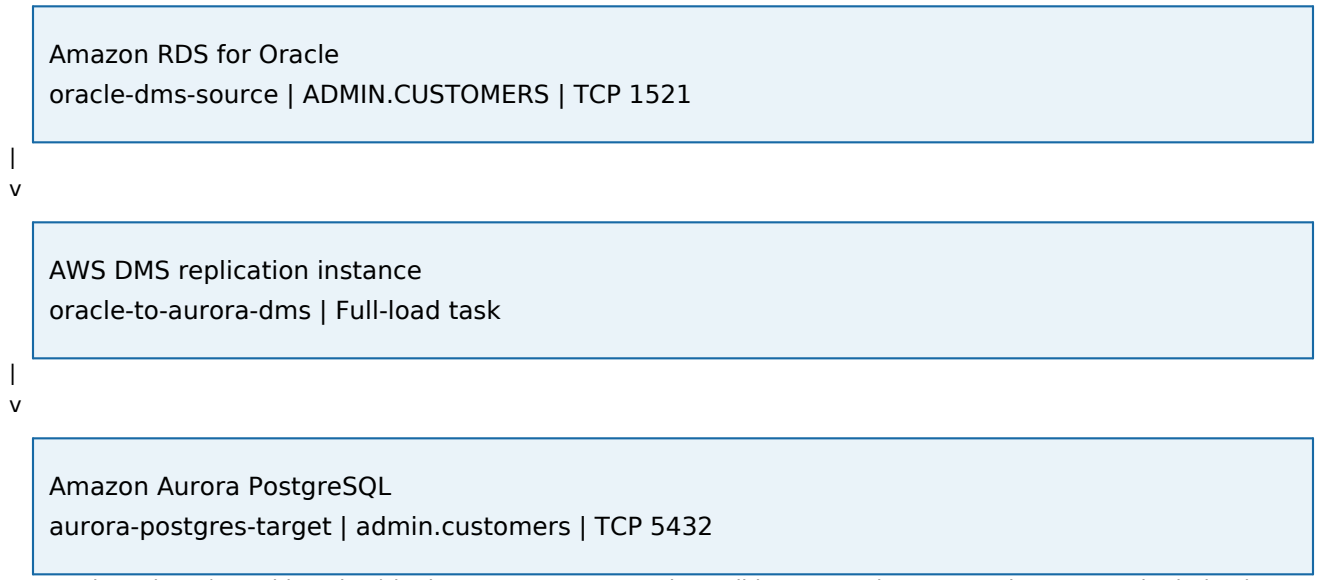
Portfolio case study based on the completed hands-on lab. October 2026.

Scope note: The exercise migrated one example table, not an entire production Oracle schema. Schema conversion for this table was performed manually; AWS DMS moved the records.

1. Executive summary

This project demonstrates migrating customer data from an Oracle database hosted on Amazon RDS to an Amazon Aurora PostgreSQL-compatible database. An EC2 Linux jump host was used for SQL*Plus and psql connectivity, while AWS Database Migration Service (DMS) performed a one-time full-load migration. The source Oracle CUSTOMERS table contained five sample records, and the user confirmed all five were copied to PostgreSQL.

2. Solution architecture



An EC2 jump host (mysql-jumpbox) in the same VPC was used to validate network access and connect to both databases. The DMS instance required its own permitted network paths to the database endpoints.

3. AWS resources

Component	Resource / setting
AWS region	us-east-1 (N. Virginia)
Oracle source RDS	oracle-dms-source
Aurora PostgreSQL cluster	aurora-postgres-target
Aurora writer	aurora-postgres-target-instance-1
EC2 jump host	mysql-jumpbox (Amazon Linux 2023, x86_64)
DMS replication instance	oracle-to-aurora-dms
DMS task observed	oracle-dms-source-task
VPC used during lab	vpc-0b238388b2946a52c

4. Source and target schema

Oracle source: ADMIN.CUSTOMERS. PostgreSQL target: admin.customers. PostgreSQL identifiers were created in lowercase and DMS table mappings were used to target the corresponding objects.

```
Oracle data types: NUMBER(10), VARCHAR2(n), DATE, NUMBER(12,2)
PostgreSQL data types: INTEGER, VARCHAR(n), DATE, NUMERIC(12,2)
```

The PostgreSQL target table was prepared before the full load. DMS target table preparation was set to Do nothing to preserve the existing table.

5. Representative SQL

Oracle source table definition:

```
CREATE TABLE customers (
customer_id NUMBER(10) PRIMARY KEY,
first_name VARCHAR2(50), last_name VARCHAR2(50),
email VARCHAR2(100), city VARCHAR2(50),
state VARCHAR2(2), signup_date DATE,
annual_income NUMBER(12,2)
);
```

Aurora PostgreSQL target table definition:

```
CREATE TABLE admin.customers (
customer_id INTEGER PRIMARY KEY,
first_name VARCHAR(50), last_name VARCHAR(50),
email VARCHAR(100), city VARCHAR(50),
state VARCHAR(2), signup_date DATE,
annual_income NUMERIC(12,2)
);
```

6. Implementation steps

1. Provision source - Created or reused an RDS Oracle instance and connected from EC2 using Oracle Instant Client SQL*Plus. Populated ADMIN.CUSTOMERS with five test records.

2. Prepare target - Confirmed the Aurora PostgreSQL cluster and writer instance were available. Installed psql on EC2, connected to the postgres database, and created the lowercase admin schema/table.

3. Configure network - Added narrowly scoped database security-group inbound rules to allow access from the EC2 security group. Confirmed Oracle TCP 1521 and Aurora PostgreSQL TCP 5432 connectivity.

4. Provision replication - Created a provisioned DMS replication instance in the database VPC and selected a compatible subnet group.

5. Configure endpoints - Created source and target DMS endpoints and successfully tested both connections.

6. Create full-load task - Configured a task for migrating existing data, with table mappings for Oracle ADMIN.CUSTOMERS and the PostgreSQL admin.customers destination. The table was already created on the target.

7. Start and validate - Started the DMS task and confirmed five Oracle records were copied into PostgreSQL.

7. Example DMS table mappings

The following is a reference configuration illustrating source selection and lowercasing of the schema, table and column names. Confirm the actual saved task mapping in the DMS console before publishing it as the exact executed configuration.

```
{ "rules": [
{"rule-type":"selection","rule-id":"1","rule-name":"customers",
"object-locator":{"schema-name":"ADMIN","table-name":"CUSTOMERS"},
"rule-action":"include"},
{"rule-type":"transformation","rule-id":"2","rule-name":"schema",
"rule-target":"schema","object-locator":{"schema-name":"ADMIN"},
"rule-action":"convert-lowercase"},
{"rule-type":"transformation","rule-id":"3","rule-name":"table",
"rule-target":"table","object-locator":
{"schema-name":"ADMIN","table-name":"CUSTOMERS"},
"rule-action":"convert-lowercase"},
{"rule-type":"transformation","rule-id":"4","rule-name":"columns",
"rule-target":"column","object-locator":
{"schema-name":"ADMIN","table-name":"CUSTOMERS","column-name":"%"},
"rule-action":"convert-lowercase"}
]}
```

8. Validation and results

The reported migration outcome was five records copied from Oracle to PostgreSQL. Use the following queries to retain independently verifiable evidence in the GitHub repository.

```
Oracle SQL*Plus:
SELECT COUNT(*) FROM customers;

PostgreSQL psql:
SELECT COUNT(*) FROM admin.customers;
SELECT * FROM admin.customers ORDER BY customer_id;
```

Validation check	Result
Oracle source record count	5 (confirmed during source preparation)
PostgreSQL target record count	5 (user-reported successful copy)
DMS endpoint tests	Both successful
Data comparison of all fields	Recommended additional check

9. Troubleshooting and lessons learned

EC2 SSH connection failed: The browser-based EC2 Instance Connect service required an appropriate inbound SSH rule. After adjusting access, the instance became reachable.

Oracle ORA-12170 timeout: Oracle RDS security-group rules did not initially permit the jump host. Adding access to TCP 1521 from the EC2 security group resolved the network path.

Aurora TCP timeout: The PostgreSQL security group initially blocked TCP 5432 from EC2. After adding a source-security-group rule, Ncat connected successfully.

Missing client utilities: Installed Oracle Instant Client SQL*Plus, PostgreSQL psql, and nmap-ncat on Amazon Linux 2023.

Premigration assessment warning: DMS reported it could not create an associated premigration assessment; the task subsequently reached Start required. The user later reported successful data migration. This warning should still be investigated for production migrations.

Identifier case differences: Oracle uppercase identifiers and PostgreSQL lowercase identifiers require deliberate schema/table mapping.

10. Portfolio takeaways

This lab demonstrates cross-engine data migration, AWS networking and security-group troubleshooting, SQL client administration, target schema preparation, AWS DMS endpoint testing, full-load task execution, and migration validation. For a production-scale effort, additional work would include automated schema conversion, compatibility assessment, CDC, LOB handling, sequence and constraint review, reconciliation, and cutover planning.

11. Suggested GitHub repository structure

```
oracle-to-aurora-postgresql-dms/
README.md
docs/Oracle_to_Aurora_PostgreSQL_AWS_DMS_Project.pdf
sql/oracle_customers.sql
sql/postgres_customers.sql
dms/table-mappings.json
screenshots/ (redacted evidence)
```

Before uploading screenshots or configuration files, remove account IDs, passwords, secrets, public IPs, and other sensitive values. Review ongoing RDS, Aurora, EC2, and DMS costs and delete lab resources when no longer needed.